

Optimizing Ticket Assignment through Group Role Assignment with Agents' Busyness Degree

Evan Wells

Department of Computer Science and Mathematics
Nipissing University
North Bay, Canada
edwells522@my.nipissingu.ca

Haibin Zhu

Department of Computer Science and Mathematics
Nipissing University
North Bay, Canada
haibinz@nipissingu.ca

Abstract— The optimal assignment of Information Technology (IT) tickets is a critical solution in our ever-expanding digital age. To maximize efficiency and minimize the burden on IT staff, this article proposes a novel approach: the Stacked Role Assignment (GRAABD-WS). GRAABD-WS leverages the principles of E-CARGO, an object-oriented framework designed to solve complex systems. The core concept involves mapping the relationship between tickets and technicians, described as roles and agents, respectively. We evaluate the suitability of technicians for specific tickets by assessing their individual workloads and skill sets, resulting in an "Agent Fitness" value. This value varies based on the roles required by each ticket. By ranking technicians based on their abilities and current workloads, GRAABD-WS ensures that the most suitable technicians are assigned to each task. The effectiveness of this algorithm is demonstrated by its ability to optimally assign 11 agents to 40 tickets within 3.71 seconds. The results highlight the potential of the Stacked Role Assignment algorithm to enhance IT ticket management, providing a robust solution for dynamic and complex IT environments.

Index Terms— Information Technology (IT), Role Based Collaboration (RBC), E-CARGO, Group Role Assignment with Agents' Busyness Degree in Weighted Sum form (GRAABD-WS), Group Multi-Role Assignment (GMRA).

I. INTRODUCTION

The rapid evolution of technology has led to a surge in demand for technology services across businesses, schools, and various establishments. This surge often results in an overwhelming influx of tickets—issues or requests related to technology—into technology service departments. These tickets frequently arrive without any systematic order, creating inefficiencies. In some cases, lower-level employees are inundated with tickets and must escalate them up the departmental hierarchy until they receive directions. In others, technicians must claim tickets from a large pool, or managers must delegate all the tickets to the staff. These methods are inefficient and unsustainable in the ever-evolving technological world.

Inefficient ticket assignment processes can lead to increased resolution times, decreased customer satisfaction, and technician burnout. To address these challenges, various solutions have been proposed, including those leveraging machine learning and artificial intelligence to predict optimal ticket assignments. While these approaches have shown promise, they often include a bias and may not consider the diversity of skills required for the large range of tickets. Alternatively, we propose optimizing the ticket assignment

process using Role Based Collaboration (RBC) algorithms [14, 17].

Specifically, we introduce a variation to the Group Role Assignment with Agents' Busyness Degree in Weighted Sum form (GRAABD-WS) [18]. The primary objective of GRAABD-WS is to streamline the assignment process by considering the abilities and current workloads of technicians. This approach aims to ensure that tickets are distributed effectively and efficiently, balancing the workload and leveraging the strengths of each technician. By calculating an "Agent Fitness" value for each ticket, GRAABD-WS can dynamically assign tickets to the most suitable technicians. As a result, tickets should be completed at a quicker rate and with the most suitable agent, after considering their workload. Through numerous trial phases and different variations to the algorithm, a stacked variant of GRAABD-WS has demonstrated its capability to assign a large number of tickets optimally across agents in a relatively short time. To the authors' knowledge, this is the first work applying RBC and related models and algorithms to ticket management.

The contributions of this paper are as follows:

- 1) Proposing a solution to maximize technicians' efficiency while preventing overwork in the domain of information technology.
- 2) Using Google's OR-Tools to solve a complex system of agent interactions.
- 3) Formalization of optimal ticket assignment using stacked GRAABD-WS.

The rest of the paper is arranged as follows. In Section II, we provide a real-world scenario to explain the ticket management problem. In Section III, we recall the related work. Then in Section IV, we formally define the problem using E-CARGO. In Section V, we illustrate the solutions and experiments. In Section VI, we present the implementation of the interactive system. Next, the usability of the system is discussed in Section VII. The conclusion and future work are placed in Section VIII.

II. REAL-WORLD SCENARIO

At Enterprise X, the IT department is struggling to manage an overwhelming number of support tickets. With only three IT technicians to service the entire company, the team is often bogged down, leading to delays and inefficiencies. The daily intake of support tickets averages around 40, varying in complexity and urgency. To better manage these tickets, IT manager John decided to implement a more structured approach

to assigning tickets based on technician abilities and current workloads.

John began by evaluating the skills of his technicians in three key areas: Communication, Troubleshooting, and Coding [12]. He then assigned a proficiency score to each technician in these areas, as shown in Table I. If he were to ask the technicians' busyness degree, i.e., a numerical value, as seen in Table II, then he could further refine his approach. Lastly, to prepare himself even further for assigning tickets, he has associated tasks to the key areas he evaluated and modified his agent's proficiency scores accordingly. The results can be seen in Table III.

Currently, John manually assigns tickets based on his understanding of each technician's skills and current workload. This process is time-consuming and prone to errors, leading to inefficiencies and delayed ticket resolutions. Additionally, the subjective nature of manual assignments can result in unequal distribution of work and potential burnout of certain technicians.

Table I. Technicians' proficiency

Technicians	Communication	Troubleshooting	Coding
TechnicianA	0.72	0.88	0.0
TechnicianB	0.5	0.89	0.81
TechnicianC	0.96	0.72	0.54

**Note: The values for the technicians were determined by adding the sum of their proficiency for the key areas involved in the ticket divided by the number of key areas involved.*

Table II. Agents' busyness

Technicians	Current Ticket Amount	Estimated Time to Complete
Technician A	4	3 hours
Technician B	3	2 hours
Technician C	6	4 ½ hours

Table III. Agents' fitness to tasks

Tickets/tasks	Key areas	Technician A	Technician B	Technician C
Wi-Fi Issue	Communication Troubleshooting	0.8	0.69	0.84
Software Issue	Troubleshooting Coding	0.36	0.85	0.63
Hardware Issue	Troubleshooting	0.88	0.89	0.72

In the scenario described above, the proper steps were taken, quantifying the technicians' talent and speed, as well as their workload. This situation describes the first steps of Role Based Collaboration (RBC); however, John encounters a Group Multi Role Assignment (GMRA) problem [10, 15], that requires consideration of an additional constraint: the busyness degree of technicians. To address this, John must implement the Stacked approach to Group Role Assignment with Agents' Busyness Degree in Weighted Sum form (GRAABD-WS) [18]. This algorithm will streamline the assignment process by considering both the proficiency scores and the current workload of each technician.

III. RELATED WORK

Research in ticket assignment systems typically involves identifying the requirements for solving a ticket, selecting the appropriate group, and assigning the task. Traditional

approaches often utilize machine learning techniques such as Support Vector Machines (SVMs) and Neural Networks (NNs) to simulate managerial decision-making. These methods have shown promise in streamlining ticket assignment processes by leveraging historical data patterns to predict optimal assignments [1, 2].

Despite their successes, these machine learning methods can suffer from biases introduced during dataset training. Such biases may result in disparate ticket assignments, particularly when contextual variations exist, such as differing requirements across schools or businesses [8]. Addressing these biases is crucial for developing fair and effective ticket assignment systems.

Recent advancements aim to overcome these limitations by employing more nuanced models. The E-CARGO model, and specifically the GRAABD-WS algorithm, represents a significant advancement. Unlike traditional SVMs and NNs, the GRAABD-WS algorithm integrates various factors, including skill levels, availability, and current workloads, to make more informed and equitable ticket assignments. This model adjusts to the current IT environment, mitigating the biases and inefficiencies inherent in traditional methods [10, 15, 18].

The GRAABD-WS approach is part of a broader trend towards evolutionary algorithms that optimize ticket distribution, improving response times and overall efficiency. This model contrasts with static machine learning models by incorporating real-time adjustments and a broader range of factors, demonstrating a more balanced and adaptive approach to ticket assignment [11].

Additional research has explored related techniques and applications, such as the use of ensemble classifiers in automated helpdesk systems [6], and advancements in automating assignment problems [3, 4, 5]. These studies underscore the ongoing evolution of ticket assignment methodologies, aiming for enhanced accuracy and adaptability in dynamic environments.

In summary, while traditional machine learning methods like SVMs and NNs have made significant contributions, they are limited by biases in training data. The E-CARGO model's GRAABD-WS algorithm offers a promising alternative by integrating multiple factors and adapting to dynamic conditions, providing a more balanced and effective approach to ticket assignment [16].

IV. FORMALIZATION WITH THE STACKED GRAABD-WS MODEL

Optimizing ticket assignment can be executed by quantifying the technicians' abilities and evaluating the system with Group Multi Role Assignment (GMRA). GMRA is an essential part of RBC and its associating framework E-CARGO. To clearly comprehend the model, it is essential to understand certain concepts and definitions.

The E-CARGO model represents the following: Environment, Classes, Agents, Roles, Groups and Objects, with their respective symbols being $\mathcal{E}$, $\mathcal{C}$, $\mathcal{A}$, $\mathcal{R}$, $\mathcal{G}$, and $\mathcal{O}$. [10] These components, all of the finite sets, along with three more create a system, defined as a 9-tuple, i.e. $\Sigma ::= \langle \mathcal{C}, \mathcal{O}, \mathcal{A}, \mathcal{M}, \mathcal{R}, \mathcal{E}, \mathcal{G}, \mathbf{s}_0, \mathcal{H} \rangle$ where $\mathcal{M}$ is a set of messages, $\mathcal{H}$ is a set of users and $\mathbf{s}_0$ is the initial state of the system. The relationship between these

components is quite intuitive. For example, agents form a group, which operates within an environment, initiated by a user. When modelling an assignment problem with E-CARGO, the structure can often be abstracted to variables, vectors, or matrices.

When viewing the optimal ticket assignment problem, tickets represent roles and technicians represent agents.

Definition 1. A *role* is defined as $r ::= \langle \overline{d}_r, V_r^* \rangle$ [10, 17, 19], where $\overline{d}_r$ is the identification of the role and V_r^* expresses the requirements for agents to be able to play the role, also known as *demands*. V_r^* is a set that designates the necessary qualifications for tickets. The number of roles is designated by the variable n .

Note: In the real-world scenario, the roles are defined as tickets/tasks.

Definition 2. An *agent* is defined as $a ::= \langle \overline{d}_a, R_q \rangle$ [10, 17], where $\overline{d}_a$ is the identification of the agent and R_q is a set of the agents' proficiency to all roles. Suppose roles are numbered as j ($0 \leq j \leq n-1$), then role j associates to the value of $R_q[j]$. The number of agents is designated by the variable m .

Note: In the real-world scenario, the agents were technicians and the table from Table III demonstrates R_q . The values of $R_q \in [0, 1]$.

Definition 3. A^a is a set of *abilities* with q elements, where each ability k ($0 \leq k \leq q-1$) is a potential value for the *demands* of the role.

Note: In the real-world scenario, each row of Table I displays *abilities* for the corresponding technician.

Definition 4. A^i is a set of *ability indices* with q elements, where if each ability k ($0 \leq k \leq q-1$) is bigger than zero, $A^i[k] = 1$. Otherwise, $A^i[k] = 0$. The *ability index* is used to evaluate constraints and is defined by:

$$A^i[k] = \begin{cases} 0, & A^a[k] \leq 0 \\ 1, & A^a[k] > 0 \end{cases}$$

Definition 5. An *ability matrix* Q^A [10, 12, 16] is a $m \times q$ matrix of values in $[0, 1]$, where $Q^A[i, k]$ expresses the proficiency of agent i for ability k .

Note: In the ticket assignment scenario, Table I is a visual example of the *ability matrix*.

Definition 6. A *role range vector* denoted as L [10, 16, 18], is of q length and identifies the number of agents necessary for each *ability* k , supposing *abilities* are numbered as k ($0 \leq k \leq q-1$). The *role range vector* is decided by a categorization of the tickets.

Note: In the real-world scenario, the *role range vector* would be dictated by the specific tasks issued by the ticket. For example, an issue with the Wi-Fi would need an agent proficient in communication and troubleshooting, as can be seen in Table III. As such, L , in this case, would be defined as a vector of $[1, 1, 0]$.

Definition 7. A *total role range matrix* is an $n \times q$ matrix, denoted as L^T , where each column is the respective *role range vector* (L) for the ticket j ($0 \leq j \leq n-1$).

Note: In the real-world scenario, L^T for Table III would be as shown below (Table III is a transpose):

$$L^T = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Definition 8. A *starting busyness vector* is an m -vector, expressed by V^s . $V^s[i]$ represents a numerical value of the busyness degree of agent i before the assignment process ($0 \leq i \leq m-1$).

Note: In the ticket assignment scenario, the estimated time to complete a column of Table II is the value within V^s , creating the vector $[3, 2, 4.5]$. Priority p affects the value of V^s ($V^s \times (1 + (\frac{p}{10}))$).

Definition 9. A *potential busyness vector*, V^p , is an m -vector. $V^p[i]$ represents a numerical value of the busyness degree of any new tickets assigned to agent i ($0 \leq i \leq m-1$).

Note: In the real-world scenario, if the manager were to assign all three tickets from Table III at the same time, he would quantify the workload of each ticket on the agent before assigning the next. Priority p affects the value of V^p ($V^p \times (1 + (\frac{p}{10}))$).

Definition 10. A *busyness vector*, V^b [17] is a vector of m length where each value of $V^b[i]$ is determined by the sum of $V^s[i]$ and $V^p[i]$, normalized so that $V^b[i] \in [0, 1]$ ($0 \leq i \leq m-1$). $V^b[i]$ is defined by the following equation, giving a value to the busyness of agent i :

$$V^b[i] = \frac{(V^s[i] + V^p[i]) - \min_{0 \leq i \leq m-1} (V^s[i] + V^p[i])}{\max_{0 \leq i \leq m-1} (V^s[i] + V^p[i]) - \min_{0 \leq i \leq m-1} (V^s[i] + V^p[i])}$$

Note: This equation is normalized through Min-Max Feature Scaling. The results will cause every value in the *busyness vector* to exist between 0 and 1, where 1 being the busiest and 0 the opposite. An alternative method can also be used named Min-Max Scaling with Feature Range. This method is almost identical; however, instead of being bound between 0 and 1, its constraints are controlled between two specified values. This flexibility allows for a better fit depending on specific requirements and contexts.

Definition 11. A *role assignment matrix* (T) [10, 15] is an $m \times n$ matrix of values $\{0, 1\}$. If $T[i, j] = 1$, then agent i has been assigned to ticket j . If $T[i, j] = 0$, then the agent has not been assigned to ticket j . It is denoted as

$$T[i, j] \in \{0, 1\} (0 \leq i \leq m-1, 0 \leq j \leq n-1).$$

Note: In the real-world scenario, if all the tickets were assigned to Technician A, the *role assignment matrix* would be:

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Definition 12. A *qualification matrix*, denoted as Q is a $m \times n$ matrix of values $[0, 1]$ where $Q[i, j]$ expresses the fitness of an agent i for a ticket j ($0 \leq i \leq m-1, 0 \leq j \leq n-1$). As the *role range vector* (L) varies depending on the ticket, Q is subject to:

$$Q = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} \frac{\sum_{k=0}^{q-1} Q^A[i, k] \times L^T[j, k]}{\sum_{k=0}^{q-1} L^T[j, k]}$$

Where for every ticket j , every agent i gets their fitness by finding the sum of their role proficiencies divided by the number of roles required for the ticket j .

Note: This equation is dependent on both the proficiency of the agent for each role and the agent's current busyness degree. The proficiency of the agents is assessed by the abilities matrix

(Definition 4) for a particular role range vector L . The role range vector L is directly influenced by the task categorization of a ticket. In the real-world scenario, Table III is a visual example of the *qualification matrix* (Q), but in a transposed format, for simplicity in visualization.

Definition 13. A *weight*, denoted as $w \in [0,1]$, is a nonnegative value used to vary the pertinence of the agents' workloads. Ideally, the *weight* is very low, to give busier agents a break. However, to optimize assignment, sometimes *weight* can be raised, so that the workload is considered to a lesser degree.

Definition 14. A *group performance* σ [10, 14] is defined as the sum of assigned agents' fitness divided by the number of agents (m), $\sigma \in [0,1]$, i.e., closer to 1 means more efficient. For a specific ticket j , σ is defined by:

$$\sigma = \frac{\sum_{i=0}^{m-1} \sum_{j=0}^{n-1} Q[i,j] \times T[i,j]}{\sum_{i=0}^{m-1} T[i,j]}$$

Note: *Group performance* is measured by the ticket, and a group can consist of a single agent.

Definition 15. Given a Q , L^T , V^b and w , the *Stacked Group Role Assignment with Agent's Potential Busyness Degree* (SGRAABD-WS) aims to find a workable T such that:

$$\sigma^{bw} = \max \left\{ w \times \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} Q[i,j] \times T[i,j] - (1-w) \times \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} V^b[i] \times T[i,j] \right\} \quad (15.1)$$

Subject to:

$$T[i,j] \in \{0, 1\} \quad (0 \leq i \leq m-1, 0 \leq j \leq n-1) \quad (15.2)$$

$$V^b[i] \in [0, 1] \quad (0 \leq i \leq m-1) \quad (15.3)$$

$$\sum_{i=0}^{m-1} T[i,j] = \sum_{k=0}^{o-1} L^T[j,k] \quad (0 \leq i \leq m-1, 0 \leq j \leq n-1, 0 \leq k \leq t-1) \quad (15.4)$$

$$\sum_{i=0}^{m-1} T[i,j] \times A^i[k] \geq L^T[j,k] \quad (0 \leq i \leq m-1, 0 \leq j \leq n-1, 0 \leq k \leq t-1) \quad (15.5)$$

where expression (15.2) is an integer constraint that limits the *role assignment matrix*. (15.3) is a constraint on the *busyness vector* that is limited to the span of 0 to 1, (15.4) causes the total amount of agents assigned for a ticket to be equal to the number of roles required in the *role range vector* and (15.5) ensures that for each ability required for the ticket, there is an agent with a non-zero proficiency to it.

The stacked GRAABD-WS algorithm extends the original GRAABD-WS algorithm by incorporating L^T , V^b and V^p . The additional members allow for multiple simultaneous assignments, taking all variables into account.

Based on the formalization stated above, the optimization of ticket assignments can be defined as a linear programming problem, which can be solved using an optimization tool. In this case, we utilize Google's OR-Tools in Java [7].

V. THE OR-TOOLS SOLUTION AND EXPERIMENT

To verify the productive functionality of the SGRAABD-WS algorithm, multiple scenarios were considered using a varying number of agents and tickets. The data is based on the influx of tickets a school board receives daily. For this school board, there are 11 technicians, with their proficiency for roles determined by their respective positions, and an influx of roughly 40 tickets a day. The time to complete a ticket varies from fifteen minutes for basic tasks, up to three hours for the more complex ones. Tickets are also assigned a certain priority 1-5 to account for the large difference of importance between

students, teachers and administrative staff. Using the provided data, each ticket can be evaluated and assigned to the optimal choices.

Before evaluating the data, there are a few necessary preprocessing steps. Firstly, it is necessary to calculate the agent's busyness degree, i.e., V^b . To achieve this, Definitions 7, 8 and 9 can be used to create a vector of agents' busyness, quantifying their tickets, scaling to priority to give time to more important tickets. The summation of V^p and V^s can be normalized to create the busyness vector from Definition 9.

Afterwards, the *total role range matrix* L^T (Definition 6) must be created for each ticket, using the task related to the ticket, as mentioned in Definition 3. The task has a direct relation to a *role range vector*, L , which is what decides the necessary roles for a ticket, as shown in Definition 5. The *role range vectors* for the tickets constitute the rows of L^T to obtain the *total role range matrix*, a key component of the assignment process.

Next, using the obtained L^T of the previous step, Q can be calculated. According to Definition 11, a *qualification matrix* (Q) can be produced by calculating the sum of the necessary roles from the *abilities matrix* (Definition 4) and dividing them by the sum of the *role range vector* for the ticket. The necessary roles are dictated by the *role range vector*, based on Definition 4. Each row of Q represents fitness for a specific ticket. The *role range vector* is determined by each row of L^T . Lastly, by Definition 12, there is the *weight* to determine. By default, it is a relatively low value and increments upwards until a solution is found. For this experiment, it begins at 0.5.

After the steps above have been completed, all the necessary values to solve the system and evaluate optimal assignment will be found. These values, Q , V^b and w with the constraints enforced by L^T constitute all necessary components to find σ^{bw} , according to Definition 14. With the aid of Definitions 10 and 13, we can come to a solution to this linear programming problem with Google's OR-Tools.

Table IV. Hardware and Software Used for the Experiment

Hardware	
CPU	11th Gen Intel(R) Core(TM) i9-11900KF 3.50GHz
Memory	32.0GB DDR4
Software	
OS	Windows 10 Pro
Editor	Visual Studio Code 1.91.1
Java	JDK 22

We can first simulate the model usage in an isolated environment to demonstrate the running results of the program with a large amount of arbitrarily scaled agents, handling a considerable number of tickets. For the sake of having bounds, we will limit the number of agents to 50 and the number of tickets to 200, with 8 potential abilities required for the tickets, to represent a significant need for optimized ticket assignment. For this isolated experiment, the time for each ticket was set to a random value between 0.25 – 3 and the number of abilities (A^i) is 8. Through 100 iterations of the model, both the *group performance* (σ) and processing time stay consistent through iterations. Figs. 1 and 2 show the results of each step of the simulation process.

In Fig. 2, even with the proficiency being set as a random decimal value bound by 0 and 1, the group score (σ) stayed

consistent throughout the simulations. The value seems to stay within the span of 0.85 and 0.95, as can be seen on the ladder of the Y-axis. As for Fig. 1, the computational speed of the model is quite fast, considering that 200 tickets are being assigned, with 50 agents to choose from. The time to completely evaluate the system is always below a second. As for scaling, with 40 tickets and 11 agents, the average time to complete was about 100ms. It can be seen that the scaling of the algorithm is quite consistent.

In reality, this model is used within some sort of user interface, not an isolated environment. In this case, it is used in a ticket management system, where the managers themselves choose the tasks related to each ticket, the number of roles and agents required for each task (L) and the *abilities* of each agent (Q^A). Within this environment, completely controlled by the manager, we can return to the initial scenario. 11 agents, whose abilities are scaled from their position, must be assigned 40 tickets daily. With this in mind, we can create a real set of data to reflect this requirement.

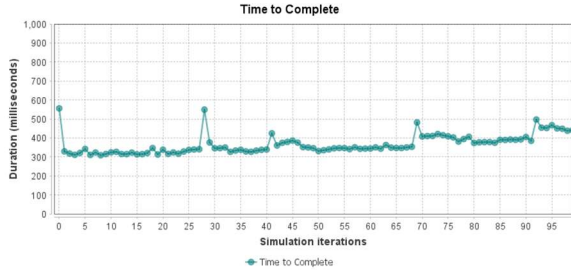


Fig. 1. Graph of times to complete for the GMRA algorithm over 100 simulations

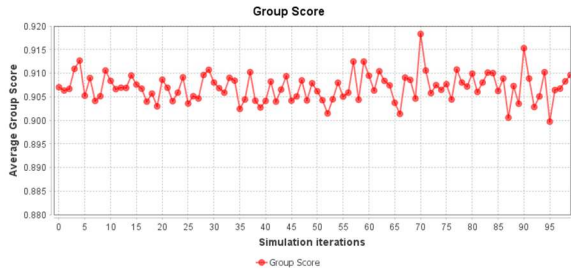


Fig. 2. Graph of group score for the GMRA algorithm over 100 simulations

Ticket	agent1	agent2	agent3	agent4	agent5	agent6	agent7	agent8	agent9	agent10	agent11
Blue call	0.8	0.7	0.7	0.8	0.7	0.8	0.5	0.8	0.4	0.8	0.8
Need not password	0.8	0.4	0.5	0.8	0.7	0.8	0.5	0.5	0.3	0.8	0.8
Virus	0.8	0.4	0.5	0.8	0.7	0.8	0.5	0.75	0.9	0.8	0.8
Cracked screen	0.8	0.7	0.7	0.8	0.7	0.8	0.5	0.8	0.4	0.8	0.8
Need login credentials	0.8	0.8	0.9	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Forgot password	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Mouse not turning on	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Fan not working	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Chrome not responding	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
New laptop?	0.7	0.8	0.7	0.8	0.4	0.8	0.8	0.8	0.8	0.8	0.8
Slow net	0.8	0.7	0.7	0.8	0.7	0.8	0.5	0.8	0.4	0.8	0.8
Keyboard not working	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Chrome freezing	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Forgot password	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Laptop is burning hot	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
I want a new iPhone	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Can't use printers	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Won't let me get in password	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Laptop screen cracked	0.8	0.7	0.7	0.8	0.7	0.8	0.5	0.8	0.4	0.8	0.8
I think I have a virus	0.8	0.4	0.5	0.8	0.7	0.8	0.5	0.75	0.9	0.8	0.8
PHP updating slow	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Installing Microsoft suite	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Need access to printers	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.8	0.4	0.8	0.8
Print case broken	0.8	0.7	0.7	0.8	0.7	0.8	0.5	0.8	0.4	0.8	0.8

Fig. 3. The SGRAABD-WS results

After simulating 40 tickets, the optimized assignment choices are displayed in an interactive table, as illustrated in Fig. 3. Note that the display shows a transpose of the matrix Q as the number of tickets is dynamic and is most likely much

bigger than the number of agents. The bolded entries represent the optimal assignment choices while the red border designates the final assignment decision. By default, the optimal assignment is selected as the final assignment, but this decision can be overridden by a manager, as can be seen in the first row of Fig. 3. The bolded entries (agents 8 and 11) were not selected as the final assignment. This manual overriding leads to adjustments that are made from subjective insights or considerations from the manager, to prevent conflicts or simply off agent availability, such as an absence. While manual overrides can lead to a slightly lower group performance score (σ), they ensure a more nuanced and context-aware assignment process, balancing efficiency with practical considerations. This table, optimized through SGRAABD-WS, was fetched, evaluated and displayed on an average of 3.71 seconds (for 11 agents and 40 tickets).

The ticket management system's user interface supports real-time adjustments, allowing managers to: review optimal assignments, make manual overrides and monitor performance metrics, with information on the agents and tickets involved. By providing this flexibility, the system ensures that ticket assignments are not only optimized for efficiency but also adaptable to the dynamic needs of the school board's operations.

VI. IMPLEMENTATION

Throughout the implementation process, the Role Based Collaboration (RBC) model [16] was used when designing the user interface. This model, as seen in Fig. 4, provides all the necessary steps to properly assign tickets efficiently.

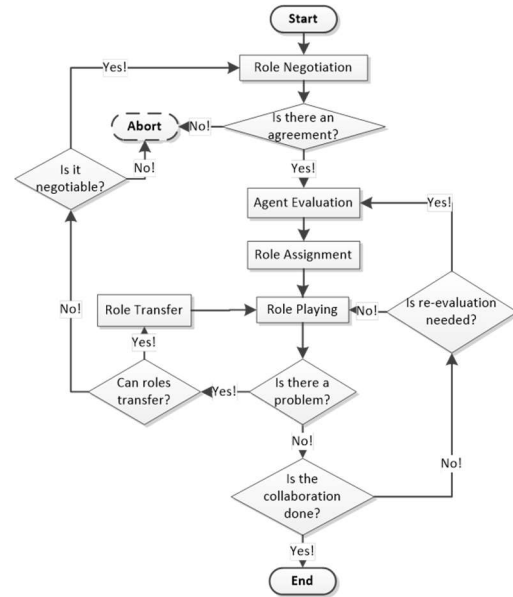


Fig. 4. The process of RBC [9, 10]

The management has the option to change the abilities required (role range vector) for tickets as they see fit, as seen in Fig. 5, where the top shows the current necessary abilities, and the bottom changes the values of the role range vector (L).

A. Agent Evaluation

Agent evaluation is done when creating an agent, where the manager is responsible for ranking the agents' abilities. In addition, the manager can raise or lower all relevant abilities, as

shown in Fig. 6. Only non-zero ability values can be scaled; the other abilities need to be added independently.

B. Role Assignment

Role assignment is the process of evaluating agents with SGRAABD-WS and returning optimal assignment results, with the tickets being “roles”. As such, the results of this step can be seen in Fig. 3, from the solution section.

C. Role Playing

Role playing is the actual processing of the assigned tickets. However, within role playing, there is a chance to transfer or re-evaluate an assignment (refer to Fig. 4). These steps may be necessary for tickets that have not been solved over a long period of time. For example, Fig. 7 demonstrates transferring agents, where agents can be added or removed by managers. In addition, there is a reassignment process (Fig. 8) which undergoes reevaluating agents’ abilities for a specific role (ticket) and allowing a visualization of the Q^4 matrix for assignment, instead of Q . In sum, the user interface allows an in-depth utilization of the RBC lifecycle.

This is the current role assignment for this type of task

Role Name	Amount
IT Support	1

Current Estimated Time:
1:0

Modify the role assignment and estimated time as necessary for your enterprise

Modify Role Assignment

IT Support
Quantity:

Database Administration
Quantity:

Network Specialist
Quantity:

Systems Administration
Quantity:

Software Specialist
Quantity:

Hardware Specialist
Quantity:

Hardware Support
Quantity:

Supervisor
Quantity:

Estimated Time
Estimated Time (hours):

Fig. 5. Screenshot of modifying ticket requirements



Fig. 6. Screenshot of scaling agents’ abilities

Agent Change

Select Agent:

Select Role:

Current Agents and Roles:
agent2 - IT Support

Fig. 7. Screenshot of Role Transferring for tickets

Pick who you'd like to assign the tasks to. Bolded entries are the optimal choices considering the agents' busyness and skillsets

Agent	IT Support	Database Administration	Network Specialist	Systems Administration	Software Specialist	Hardware Specialist	Hardware Support	Supervisor
agent1	0.8	0.0	0.0	0.3	0.0	0.0	0.0	0.6
agent2	1.0	0.0	0.4	0.5	0.0	0.0	0.0	0.8
agent3	0.9	0.0	0.5	0.0	0.0	0.0	0.0	0.5
agent4	0.0	0.3	0.8	0.9	0.3	0.0	0.0	0.0
agent5	0.0	0.4	0.7	0.0	0.0	0.8	0.8	0.4
agent6	0.0	0.6	0.0	0.5	0.8	0.0	0.0	0.0
agent7	0.0	0.0	0.5	0.6	0.0	0.5	0.5	0.0
agent8	0.0	0.9	0.8	0.0	0.7	0.0	0.0	0.0
agent9	0.4	0.3	0.0	0.0	0.0	0.2	0.2	0.7
agent10	0.0	0.6	0.0	0.5	1.0	0.0	0.0	0.0
agent11	0.9	0.3	0.0	0.0	0.0	0.0	0.0	0.7

Fig. 8. Screenshot of Ticket Reassignment

VII. USABILITY

The usability testing of the ticket management system revealed key insights from IT staff members who tried the prototype. The prototype garnered positive feedback from IT staff, especially for its balance between workload and agent proficiency and the assignment of tickets based on requirements. The manager dashboard was lauded for its comprehensive overview of all agents, tickets, and roles, offering a complete snapshot of the entire IT department. This holistic view allowed managers to monitor operations effectively. However, improvements were suggested regarding the ability for agents to refuse or request removal from tickets, particularly when unqualified or facing task conflicts. Adding features to accommodate ticket refusal and reassignment was suggested as essential for further refinement. Such enhancements could provide better user control and improve overall satisfaction. Additionally, agents noted that having more flexibility in managing assignments would alleviate unnecessary pressure and increase efficiency.

Other suggestions included enhancing the visual representation of ticket prioritization and progress tracking to further optimize usability. Incorporating these features into the next iteration would not only refine the interface but also ensure a more streamlined workflow, better aligning tasks with agent capabilities and departmental objectives.

VIII. CONCLUSIONS

In sum, the contribution of this paper lies in the proposed solution for optimizing IT ticket assignment. Its basis consists of evaluating agents’ skills and their workloads, using the data and the unique category-subcategory mapping to nominate the most fit agents to the tasks. To the authors’ knowledge, this is the first work in applying RBC/E-CARGO and related optimization models to ticket management.

Utilizing this optimization method, IT managers can utilize the RBC and GRA models to augment efficiency within their department and lessen the load of work of their staff. The

effectiveness of this work has been proven through numerous sets of simulations, showing consistently high group scores.

Future work on this issue might address:

- 1) Establishing relations between agents and roles to account for concepts such as conflicts and cooperation [9, 13]. These concepts will lead to further optimization of ticket assignment.
- 2) Further researching other components that can impact an agents' qualification, other than their workload.

ACKNOWLEDGEMENT

This work is supported in part by NSER Discovery Grant (RGPIN 04818-2018), SSHRC Insight Grant (IG 435-2023-1056), and NSERC Discovery Development Grant (DDG-2024-00036).

REFERENCES

- [1] S. Agarwal, J. Bandlamudi, A. Mandal, A. Ray, and G. Sridhara, "Automated Assignment of Helpdesk Email Tickets: An AI Lifecycle Case Study," *AI Magazine*, vol. 41, no. 3, pp. 45-62, Sept. 2020.
- [2] S. Agarwal, R. Sindhgatta, and B. Sengupta, "SmartDispatch: Enabling Efficient Ticket Dispatch in an IT Service Environment," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, pp. 1393-1401, 2020.
- [3] N. Arici, L. Putelli, A. E. Gerevini, L. Sigalini, and I. Serina, "LLM-based Approaches for Automatic Ticket Assignment: A Real-world Italian Application," in *Proceedings of the Seventh Workshop on Natural Language for Artificial Intelligence (NL4AI 2023)*, Rome, Italy, Nov. 6-7, 2023.
- [4] H. S. Gupta and B. Sengupta, "Scheduling Service Tickets in Shared Delivery," in *Service-Oriented Computing. ICSOC 2012. Lecture Notes in Computer Science*, vol. 7636, C. Liu, H. Ludwig, F. Toumani, and Q. Yu, Eds. Berlin, Heidelberg: Springer, 2012, pp. 84-98.
- [5] M. Krishnan, M. K. Srinivasan, and C. A. Mathews, "Agent Score-Based Intelligent Incident Allocation Engine," in *ICT Analysis and Applications*, S. Fong, N. Dey, and A. Joshi, Eds. Singapore: Springer, 2021, vol. 154, pp. 295-307.
- [6] A. Mandal, N. Malhotra, S. Agarwal, A. Ray, and G. Sridhara, "Cognitive System to Achieve Human-Level Accuracy in Automated Assignment of Helpdesk Email Tickets," in *Service-Oriented Computing: 16th International Conference, ICSOC 2018, Hangzhou, China, November 12-15, 2018, Proceedings*, vol. 16, pp. 332-341. Springer International Publishing, 2018.
- [7] L. Perron and V. Furnon, "OR-Tools," v9.10, Google [Online]. Available: <https://developers.google.com/optimization/>, May 7, 2024.
- [8] A. Zangari, M. Marcuzzo, M. Schiavinato, A. Gasparetto, and A. Albarelli, "Ticket automation: An insight into current research with applications to multi-level classification scenarios," *Expert Systems with Applications*, vol. 225, p. 119984, Sept. 2023.
- [9] H. Zhu, "Avoiding Conflicts by Group Role Assignment," *IEEE Trans. on Systems, Man, and Cybernetics: Systems*, vol. 46, no. 4, April 2016, pp. 535-547.
- [10] H. Zhu, *E-CARGO and Role-Based Collaboration: Modeling and Solving Problems in the Complex World*, Wiley-IEEE Press, NJ, USA, Dec. 2021.
- [11] H. Zhu and L. Feng, "Agent Evaluation in Distributed Adaptive Systems," *The IEEE Int'l Conf. on Systems, Man and Cybernetics (SMC'13)*, Manchester, UK, Oct. 13-16, 2013, pp.752-757.
- [12] H. Zhu, and M. Grenier, "Agent Evaluation for Role Assignment," in *Proc. of the IEEE 8th Int'l Conf. on Cognitive Informatics (ICCI'09)*, Hong Kong, China, 2009, pp.405-411.
- [13] H. Zhu, "Group Multi-role Assignment with Conflicting Roles and Agents," *IEEE/CAA J. of Automatica Sinica*, vol. 7, no. 6, Nov. 2020, pp. 1498-1510.
- [14] H. Zhu, M. Hou, C. Wang, and M.C. Zhou, "An Efficient Outpatient Scheduling Approach," *IEEE Trans. on Automation Science and Engineering*, vol. 9, no. 4, Oct. 2012, pp. 701-709.
- [15] H. Zhu, D. Liu, S. Zhang, S. Teng, and Y. Zhu, "Solving the Group Multi-Role Assignment Problem by Improving the ILOG Approach," *IEEE Trans. on Systems, Man, and Cybernetics: Systems*, vol. 47, no. 12, Dec. 2017, pp. 3418-3424.
- [16] H. Zhu, M.C. Zhou, and R. Alkins, "Group Role Assignment via a Kuhn-Munkres Algorithm-based Solution," *IEEE Trans. on Systems, Man, and Cybernetics, Part A: Systems and Humans*, vol. 42, no. 3, May 2012, pp. 739-750.
- [17] H. Zhu, and M. Zhou, "Role-Based Collaboration and its Kernel Mechanisms," *IEEE Trans. on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, vol. 36, no. 4, July. 2006, pp. 578-589.
- [18] H. Zhu, and Y. D. Zhu, "Group Role Assignment with Agents' Busyness Degrees," *The IEEE Int'l Conf. on Systems, Man and Cybernetics (SMC'17)*, Banff, Canada, Oct. 5-8, 2017, pp. 3201-3206.
- [19] X. Zhu, H. Zhu, D. Liu, and X. Zhou, "Criteria Making in Role Negotiation," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 50, no. 10, Oct. 2020, pp. 3731-3740.